JIT Compilation for User-Defined Functions

MCDS Capstone Final Report
July 16, 2022

Kyle Dotterrer, Andrew Pavlo
Carnegie Mellon University
{kdottere, pavlo}@cs.cmu.edu

Abstract

User-defined functions (UDFs) are a powerful tool for expressing complex application logic in SQL queries. Although they offer a practical complement to SQL, their use in production database environments is often limited because of their associated performance costs [27, 14]. In this project, we evaluate the efficacy of methods intended to address the performance limitations of UDFs in the context of database management systems (DBMSs). We compare the performance of an existing, state-of-the-art technique, UDF inlining, with our own novel strategy that executes UDFs as native code via JIT-compilation. In an experimental evaluation, we find that UDF JIT-compilation and UDF inlining provide similar levels of performance benefits.

1. INTRODUCTION

User-defined functions (UDFs) are a powerful tool for encapsulating complex application logic that may subsequently be embedded into SQL queries. Consider a scenario in which we would like to write a query that computes the "reward level" for each customer in a table of retail transactions based on the total amount spent by each customer. Expressing such a query in pure SQL is possible, but non-trivial. In contrast, with the help of SQL's procedural extensions, the definition of a UDF that implements this logic is straightforward, as shown in figure 1.

```
CREATE FUNCTION cust_level(ckey INT)
    RETURNS CHAR(10) AS $$
    DECLARE total FLOAT;
    DECLARE level CHAR(10);
BEGIN
    total = (SELECT SUM(o_totalprice)
        FROM orders WHERE o_custkey=ckey);
    IF (total > 1000000)
        level = 'Platinum';
    ELSE
        level = 'Regular';
    END IF;
    RETURN level;
END
$$ LANGUAGE PLPGSQL;
```

Listing 1: The definition of UDF **cust_level**.

Once the UDF is defined, we can then use the UDF in subsequent queries to efficiently express high-level application logic. A query that invokes the UDF **cust_level** is shown in figure 2.

```
SELECT c_custkey, cust_level(c_custkey)
    FROM customer;
```

Listing 2: The invocation of UDF **cust_level**.

This simple example demonstrates the power of user-defined functions. UDFs allow us express SQL queries more efficiently and in a manner that encourages modularization and reuse. However, UDFs present one major limitation that often precludes their use in production systems: they introduce an intolerable amount of runtime overhead. One study from Microsoft found that introducing UDFs to express portions of query logic can increase total execution time by over 46,000x its baseline value when expressed in pure SQL [27]. The underlying reason for this massive performance degradation is the fundamental mismatch between declarative SQL and the imperative language of UDFs. Because the source languages are so different, most database systems rely on a distinct subsystem to implement UDF execution, and the context switching between these subsystems combined with the lack of optimization applied within the UDF engine produces this impressively-poor performance [27, 14].

Various approaches to addressing this problem have been proposed. The most promising existing strategies for improving UDF execution performance may be approximately grouped under the term *UDF inlining*. In UDF inlining, the imperative code that implements the UDF is translated to SQL, and the resulting SQL is then inserted into the calling query at the invocation site of the UDF. In this project, we propose a new approach to UDF execution that operates by leveraging an existing code generation system to JIT compile the UDF to native machine code and subsequently execute it. We implement this technique in the NoisePage [6] DBMS.

The remainder of this document is structured as follows. In Section 2 we provide background on query compilation and UDF compilation for analytics. In Section 3 we survey related work on UDF runtime optimization, focusing on approaches to UDF inlining. In Section 4 we describe the architecture of the query execution engine in NoisePage and in Section 5 we describe our extensions to support UDF compilation. Section 6 presents the results of our experimental evaluation. We discuss directions for future work in Section 7 and conclude in Section 8.

2. BACKGROUND

In this section, we provide general background on the technical concepts relevant to our project. The two topics of interest include query compilation in the context of database management systems and UDF compilation in distributed analytics frameworks.

2.1 Query Compilation

In traditional disk-oriented systems, the DBMS implements query execution entirely in software. The canonical execution strategy employed by many database systems begins as the optimizer converts the abstract syntax tree produced by parsing the query text into a physical plan tree. Each node in this plan tree represents a physical operator that implements one or more logical operations specified in the input query, such as a **SELECT** operation, application of a filter (**WHERE**), or a **JOIN**. With the plan tree constructed, the DBMS executes the query by evaluating each operator in the physical plan [16, 17, 29].

In this strategy, the DBMS implements its execution engine in terms of high-level, general-purpose code. This degree of generality is excellent from a software engineering perspective because it makes query execution easy to reason about. However, this comprehensibility comes at the cost of runtime performance. The generality of the code in this execution technique imposes runtime overhead by way of the many levels of indirection that it introduces, resulting in a large number of native instructions being executed [15]. The number of executed instructions is often not a limiting factor in disk-oriented systems' performance because disk IO operations dominate the total query processing time. For in-memory systems where persistent storage no longer imposes this bottleneck, several strategies are available to reduce the number of instructions executed in query processing.

Query compilation is one such query execution strategy that is growing in popularity. At a high level, query compilation aims to reduce the number of instructions executed by generating specialized code for each query that the system encounters.

There are many approaches to implementing query compilation, and the techniques for doing so have evolved rapidly in the last decade. One of the earliest implementations of query compilation in the modern era was accomplished by the HIQUE system [13]. In HIQUE, the execution engine first translates an input query to C++ code. It subsequently compiles this C++ code to native code by spawning an external process (the C/C++ compiler, GCC [10]) [22]. It is important to note here, however, that HIQUE does not implement translation for arbitrary SQL queries but instead relies on pre-defined C++ templates that it applies to a limited subset of input queries.

More advanced systems eschew use of an external compiler to avoid the high costs associated with compilation under this model. Spawning an external compilation process introduces the latency associated with operating system process management into the critical path of query execution. Furthermore, utilizing an external compiler like GCC to compile specialized query code discards much of the information that the DBMS has already 'learned' about the query while constructing its query plan and forces the compiler's frontend to repeat much of this work. The HyPer main memory DBMS [3] is one example of a system that avoids these issues by implementing query compilation without using an external compiler. HyPer generates LLVM [4] intermediate representation from the input query and subsequently utilizes the existing LLVM infrastructure to JIT-compile the IR to native machine code that implements the query [25, 21]. HyPer also adds support for *adaptive query compilation* to further reduce query execution latency. Query execution is initiated on an interpreter while fragments of the query are JIT-compiled asynchronously in the background; when compilation completes the compiled fragments are dynamically swapped with their interpreted counterparts to increase steady-state throughput.

The SingleStore [9] execution engine represents a more advanced approach to compilation. It first translates SQL queries to an internal domain-specific language (DSL) that then undergoes a second stage of translation to LLVM intermediate representation. The execution engine then JIT-compiles this intermediate representation to native code [26]. This additional step in the query compilation process facilitates greater developer productivity by making it easier to reason about the code generated by the execution engine's upper layers.

Carnegie Mellon Database Group's NoisePage system implements an approach to query compilation that is similar to SingleStore's. The first step in NoisePage's query processing pipeline is translation of the input query to an internal domain-specific language. The execution engine then compiles this DSL to a lower-level bytecode representation. Once this bytecode is generated, the NoisePage execution engine can either run the bytecode directly on an interpreter or translate this bytecode to LLVM IR that is then JIT-compiled to efficient machine code [24].

2.2 UDF Compilation for Analytics

Outside of the database systems community, the term "user-defined function" has a more general definition. In analytical applications, a user-defined function is any functionality that is not built into the processing engine itself, and is instead written by an end-user. In recent years, interest in improving the performance of user-defined functions in analytics engines has grown substantially, likely as a consequence of the growing volume of data that must be processed efficiently.

One of the earliest distributed analytics engines to adopt JIT-compilation is Apache Spark [1]. The Databricks team introduced JIT-compilation to Spark in 2016 with their Tungsten 2.0 engine [28]. This engine generates special-purpose Java Virtual Machine (JVM) bytecode for select operators in the Spark execution plan. A team of academics from Purdue University improved upon this design in 2018 with their *Flare* extension to Spark [18]. Instead of compiling Spark operators to JVM bytecode, *Flare* emits native machine code for even greater savings in instruction counts and overall runtime.

The *Tupleware* system targets a different set of users by scaling down Spark for "medium-size" datasets and analytics workloads. Where Spark targets the JVM, *Tupleware* provides a Python interface for analytics applications. The system is implemented as a Python library that allows clients to express user-defined functions in terms of a limited, functional-style API [12]. Under the hood, the system JIT-compiles these Python functions to machine code with LLVM. This system was later extended to support a wider-range of user-defined functions in the *Tuplex* system [23]. *Tuplex* improves on *Tupleware* by first profiling the dataset to collect a distribution of the types used in the workload and subsequently optimizing for the common case. At runtime, input data that does not match this expected common case falls through to a slow-path that executes on the Python interpreter.

3. RELATED WORK

In this section, we describe approaches to UDF runtime optimization. These approaches are relevant for our discussion because they are the baseline against which we measure the effectiveness of our approach.

Traditional DBMS execution engines are not equipped to execute arbitrary imperative program logic. For this reason, many systems implement a distinct execution subsystem for procedural extensions to SQL and perform a context-switch from the primary SQL execution engine to this subsystem whenever a UDF must be evaluated [14]. An example of this style of UDF evaluation is shown in figure 1.

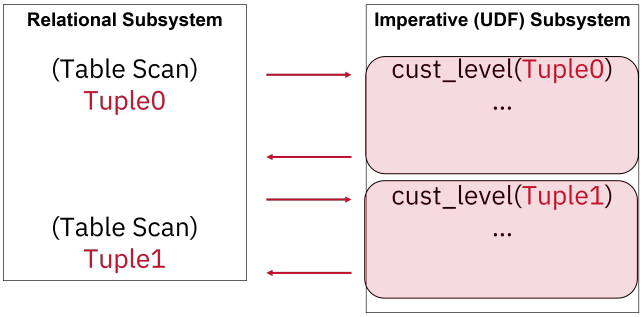


Figure 1: **Interpreted UDF Execution** – In a traditional DBMS execution engine, the SQL execution engine must context switch to a distinct execution engine for imperative code whenever it encounters a UDF.

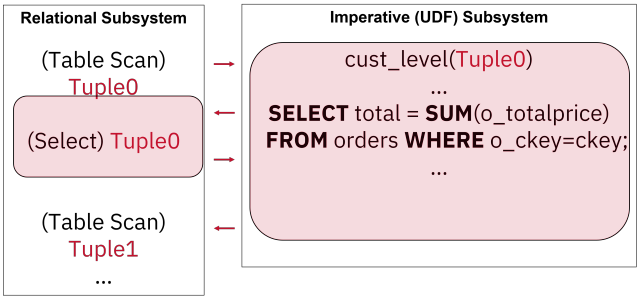


Figure 2: **Recursive Context Switching** – Most UDFs found in production workloads contain embedded SQL queries. This exacerbates performance issues because the DBMS now context switches between the SQL engine and the imperative engine repeatedly.

While this approach makes combining the two styles of execution simple to reason about, it often implies significant performance penalties. Frequent context-switching between the SQL engine and the imperative subsystem imposes huge runtime overhead. The situation is made worse by the fact that most UDFs contain embedded SQL queries, leading to further context-switches from the UDF subsystem back to the SQL engine. An example of this situation is shown in figure 2.

Despite recent advances in compilation techniques, the most successful approaches to addressing the performance limitations of user-defined functions do not involve native compilation. Instead, most of the attention in this space is currently devoted to the technique of UDF inlining. In UDF inlining, a UDF is translated into a relational algebraic expression that can then be embedded directly into the calling SQL query. Inlining a UDF allows the DBMS's existing query optimization component to optimize the logic of an imperative UDF in the same way it optimizes declarative SQL.

Ramachandra et al. implement this technique's archetype in their *Froid* system within Microsoft SQL Server [5]. *Froid* executes UDF inlining by first decomposing an individual UDF into distinct *regions* that it then translates into equivalent relational expressions. The *Froid* framework then relies on SQL Server's **APPLY** operator to compose these relational expressions into a single relational expression that encodes the entirety of the UDF's logic [27]. Gupta et al. extend this work on Froid with their *Aggify* framework to add support for iterative control-flow and cursor loops. *Aggify* enables new control flow constructs by translating loops to equivalent custom aggregations [19]. An example of the inlining technique used by *Froid* is shown in figure 3.

Duta et al. pursue a distinct approach to UDF inlining in their

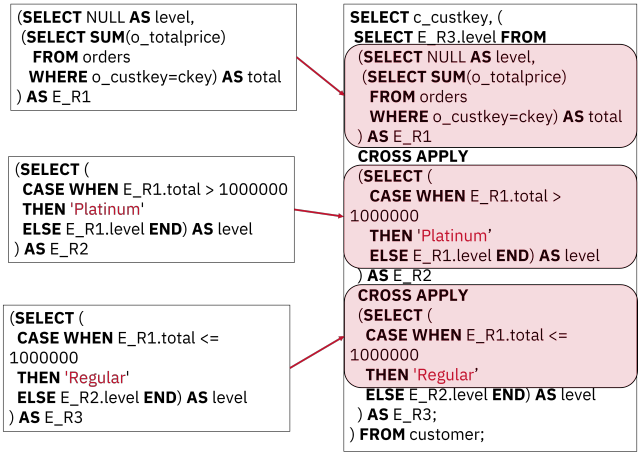


Figure 3: **UDF Inlining in *Froid*** – A simplified example of the UDF-inlining procedure implemented by the Froid framework in Microsfot SQL Server.

Apfel framework [14]. Where Ramachandra et al. implement *Froid* as an integrated component of the Microsoft SQL Server DBMS, Duta et al. design their *Apfel* framework as a standalone system that compiles imperative UDFs written in PL/pgSQL [7] to standard SQL. Any database system may then process the resulting SQL query in the usual manner. Where *Froid* relies on the **APPLY** operator, *Apfel* relies on nested subqueries combined by lateral joins (**LATERAL**) to recreate PL/SQL logic. *Apfel* leverages recursive common table expressions (SQL's **WITH RECURSIVE** syntax) to reproduce arbitrary imperative loop logic in SQL.

4. THE NOISEPAGE EXECUTION ENGINE

The architecture of the NoisePage execution engine is central to the engineering work performed in this project to add support for UDF JIT-compilation. In this section, we describe the internals of the NoisePage execution engine.

We partition the NoisePage execution engine into the following logical layers:

1. **Code Generation**
2. **Query Compilation and Execution**

We now describe each of these layers in detail.

4.1 Physical Query Plan

The entry point for the NoisePage execution engine is the physical query plan. The physical plan is the query plan produced by the query optimizer. Unlike a logical query plan, the physical plan contains all of the fine-grained query execution decisions that are required to execute the query. For example, the physical query plan explicitly states the algorithms used for **JOIN** operators and the access methods used for table scans.

4.2 Code Generation

The code generation layer of the NoisePage execution engine produces a low-level, imperative representation of a query from an input physical query plan. We transform the input query plan through several intermediate stages before emitting the highest-level abstraction that the system is capable of executing - an executable query.

We partition the process of code generation into several logical sub-phases:

1. *Pipeline Definition*
2. *AST Construction*
3. *Module Generation*
4. *Executable Query Construction*

We describe each of these sub-phases of code generation below.

4.2.1 Pipeline Definition

The *Pipline Definition* phase of code generation translates the input phsyical query plan to the *pipeline* abstraction.

We apply a critical transformation to the query abstraction model at this early stage of processing. The input to code generation is a physical query plan. Internally, this plan is represented as a tree of high-level operators that implement the query. During the initial stages of code generation, however, we break down this abstraction and replace it with the pipeline abstraction.

The pipeline abstraction was originally introduced by Neumann [25] as a means of improving query execution efficiency by exploiting microarchitectural features. Operators from the physical query plan participate in one or more pipelines. A pipeline represents a uni-directional data flow of tuples or tuple batches. Within a pipeline, operators do not materialize tuple (batch) data into cache or memory, which is the primary source of their favorable performance properties.

The procedure for constructing pipelines from the physical query plan proceeds recursively. In the initial traversal of the plan tree, for each plan node we encounter, we perform the following steps

- Construct a *translator* for the operator or expression represented by the plan node
- In the body of constructor for the translator, recursively prepare the relevant children of the plan node with which this translator is associated

A *translator* is a C++ object that is responsible for generating code for a particular query plan operator. We define a unique translator for every physical query plan node. As we descend the plan tree and produce translators for each operator or expression in the physical plan tree, we simultaneously register each translator with one or more pipelines. A translator may appear in multiple pipelines in the event that it represents a materialization point. For instance, consider the translator for the hash join operator. This translator must participate in both the pipeline that produces the tuples utilized on the "build side" of the **`JOIN`** as well as the pipeline that produces tuples utilized on the "probe side." In the NoisePage implementation, the hash join translator constructs a new pipeline with itself as the root to handle this situation.

4.2.2 AST Construction

During abstract syntax tree (AST) construction, we translate the pipeline abstraction into an AST for our internal domain-specific language (DSL), `TPL`. At this point in code generation, the physical query plan is expressed as a collection of pipelines, each of which is composed of a sequence of translators. The next step in code generation is construction of an AST from the raw pipeline abstraction. This AST subsequently serves as input to the `TPL` compiler.

In order to construct an AST from the raw pipeline abstraction, we perform the following steps:

1. Declare the top-level structures for the query. The structures are visible to all functions in the compiled query.
2. Declare top-level functions for the query. Currently, the only top-level functions produced for all queries are those required for query setup and teardown.
3. Iterate over all pipelines in the query and generate all related pipeline-level code and data. It is at this point that we perform the majority of the work of AST construction. For each pipeline, we iterate over all of the translators defined within it and invoke a set of callback functions in the appropriate order. These functions defer to logic within the translators themselves that emit high-level AST constructs into a common plan.

At the end of AST construction, we now have an AST abstraction that is ready for the `TPL` compiler. This AST is organized in much the same way that an AST for a general-purpose programming language is organized. We represent the AST as a collection of data and functions in an approximate control-flow structure that falls short of a proper control-flow graph. During the invocation of the individual translators within each pipeline mentioned above, each translator emits code into the AST at the appropriate point according to the context in which it appears. For example, a translator that appears within pipeline `Foo` emits the code that implements its functionality into the body of the `PerformPipelineWork` function for the `Foo` pipeline. Individual translators are not responsible for emitting the low-level programming constructs that manage control flow (as an example) but rather operate on high-level primitives like table iterators and vector projections.

4.2.3 TPL Module Generation

During `TPL` module generation, we transform the AST produced in the previous phase into a `TPL` module. A `TPL` module contains the bytecode program that implements the query plan.

We implement `TPL` compiler as a standalone bytecode compiler. The `TPL` compiler performs the following steps to compile the input AST to a `TPL` module:

1. Parsing: This step is elided in the event that the program is not being compiled from raw TPL source code, which is the case here because we manually constructed the AST for the program during the previous sub-phases of code generation.
2. Semantic Analysis: The input AST is traversed and checked for logical errors (e.g. type-system violations).
3. Bytecode Generation: Where the magic happens. The input AST is traversed and translated to a low-level bytecode representation that may be executed directly in a lower layer of the NoisePage execution engine.
4. Module Generation: A TPL module is constructed that assumes ownership for the raw bytecode produced in the previous step.

There is a one-to-one mapping between code and data defined in the input AST and the code and data that appears in the completed `TPL` module. There is no fundamental change to the abstraction that occurs at this stage of code generation. Instead, `TPL` module generation represents both an important lowering step (a `TPL` module is the fundamental unit of execution for interpreted execution) as well as a unification point between the more general `TPL` ecosystem and the NoisePage execution engine. The `TPL` compiler can handle both raw `TPL` programs in a textual representation as well as a previously-constructed `TPL` AST (the intermediate form produced in the previous phase of code generation). Implementing the `TPL` ecosystem in a manner that is agnostic of the NoisePage execution engine itself makes the system more amenable to development.

4.2.4 Executable Query Construction

In the final phase of code generation, we transform the TPL module into an *executable query*. The process of transforming a TPL module into an executable query is straightforward. Each executable query corresponds to a single TPL module, so we transfer ownership of the module into the executable query. This enforces a relationship in which each query corresponds to exactly one TPL module.

4.3 Query Compilation and Execution

The NoisePage execution engine supports three distinct execution modes:

- `Interpreted`: In interpreted execution, we execute the TPL bytecode program directly on an interpreter.
- `Compiled`: In compiled execution, we JIT-compile the TPL bytecode program to native machine code with LLVM [4]. We emulate the technique originally proposed by Neumann [25] to accomplish this.
- `Adaptive`: This execution mode attempts to capture the benefits of both of the above execution modes: the low-latency execution of simple queries from `Interpreted` execution and the high-throughput for complex queries from `Compiled` execution. This is accomplished by beginning interpreted execution of the query immediately while asynchronously compiling the query in the background. As compilation of query fragments completes, the compiled fragments are dynamically swapped into the running program. This technique was originally proposed by Kohn [11].

In the second logical layer of the NoisePage execution engine, we perform both interpretation of bytecode programs (in `Interpreted` execution mode) and further compilation of the byetcode program to native machine code (in `Compiled` execution mode). The interface to each execution mode supported by the execution engine is identical.

As the highest-level of query execution, we perform the same steps regardless of the execution mode:

1. Iterate through each function that composes the query and request a C++ function object that implements that function from the fragment's TPL module. The execution mode is specified in this request to the TPL module, and it is at this point that the logic of the execution engine diverges based on the execution mode in use.
2. The TPL module returns a C++ function object that implements the requested function; we push query processing forward by invoking this function object on the query state and then return to the top of the loop to request the next function from the TPL module. This process continues until all functions in the query have been executed, at which point the query is complete.

We now examine the internals of each of the available execution modes.

4.3.1 Interpreted Execution

In interpreted execution, the function object returned by the TPL module call wraps an invocation that jumps to the entry point of the NoisePage bytecode interpeter. The implementation of the NoisePage virtual machine is nicely self-contained. The VM implementation appears daunting at first (the file is nearly 3,000 lines long as of this writing, consisting almost entirely of preprocessor macros) but the actual mechanics of how the VM interprets a query are relatively straightforward.

A single function (`VM::Interpret()`) implements the core interpreter loop within NoisePage. We implement the TPL interpreter as a register-based virtual machine (VM). The VM uses indirect threaded interpretation; each bytecode handler's label is statically generated and stored in a predefined location at server compile time. Interpretation of a TPL program proceeds by reading the first instruction from the bytecode stream, executing it, and returning to the next instruction in the bytecode stream. This process may be influenced by explicit control-flow instructions.

The bytecode handlers defined for the virtual machine are only be simple register and instruction pointer manipulation instructions. The code that contains the logic of these handlers is implemented externally and invoked from the simple instructions in the byetcode stream. This requirement makes code generation to LLVM much simpler in the case that the query is natively compiled.

4.3.2 Compiled Execution

We implement the lowest-level stage of query compilation in the NoisePage LLVM engine. The LLVM engine is responsible for JIT-compiling the TPL bytecode program to native machine code. Once this process is complete, the entirety of the query executes natively on the hardware, without any additional overhead imposed by an interpreter.

Query compilation to machine code takes place in two distinct phases:

1. Lowering from the bytecode representation of the TPL program in the bytecode module to LLVM intermediate representation (IR)
2. Utilization of the LLVM JIT to compile the LLVM IR to native machine code for the target platform

As the short description above suggests, the first phase is the one we implement ourselves in NoisePage; the majority of the code in the LLVM engine is concerned with this task. Once we lower the TPL program to LLVM IR, it is a relatively simple matter to utilize the existing LLVM infrastructure to JIT this IR to native code. This is the primary purpose of the LLVM infrastructure, and also why LLVM is a common sight among DBMS execution engines that support compilation to native code.

Once the LLVM IR is compiled to native machine code, we make the native code callable from the DBMS server process itself by:

1. Loading the object code for the native machine code into an in-memory buffer
2. Converting this raw object code into an object file, loading this object file, and linking it to the address space of the DBMS server process
3. Resolving and caching each of the function definitions in the module. This step is not strictly necessary, but it elides the need to resolve procedures at runtime

Once the native code is loaded and linked to the DBMS process address space, all of the functions it exposes are accessible from regular C++ function calls. The aforementioned use of C++ function objects to wrap these calls unifies the API across calls to both interpreted and compiled functions.

5. UDF COMPILATION IN NOISEPAGE

In this section, we describe how we added support for JIT-compilation of user defined functions to the NoisePage DBMS. We partitioned the work of adding support for UDFs to NoisePage into three layers of functionality: a PL/pgSQL compiler, extension of the TPL DSL with *closures*, and execution engine integration. We describe each of these three layers in detail below.

5.1 PL/pgSQL Compiler

There are many distinct dialects of SQL's procedural extensions - different database systems choose to implement different subsets of the requirements set forth in the standard while simultaneously extending the standard functionality in various ways. In this project, we add support for the the PL/pgSQL dialect of PL/SQL. This is the procedural extensions dialect supported by the PostgreSQL database system [8]. We select PL/pgSQL as our target for a number of reasons. PostgreSQL is an open-source system, making it convenient to install and set up. It is a natural choice for performance comparisons because it implements a "textbook" database system that does not include many of the complex optimization techniques that other systems might, and makes such optimizations configurable in cases in which they are supported. Finally, PL/pgSQL is the dialect of PL SQL targeted by the *Apfel* framework, one of the targets of our performance comparison.

We implement a PL/pgSQL compiler in NoisePage. This compiler accepts the textual representation of a PL/pgSQL function as input and produces the TPL bytecode program that implements this function as output. The high-level steps involved in PL/pgSQL compilation are the following:

1. Parse the textual representation of the PL/pgSQL function to a parse tree. We utilize existing parser functionality provided by *libpg_query* for this purpose.
2. Translate the raw parse tree to an enriched abstract syntax tree. During this phase of compilation, we translate each of the nodes in the raw parse tree from *libpg_query* to an internal AST representation. This phase translates many of the unsafe components of the parse tree to safe, internal representations, such as replacing non-owning pointers to child nodes with owning C++ smart pointers and replacing string type names with enumerated type identifiers.
3. Walk the enriched AST to collect function-level metadata. This metdata includes information such as the types of all declared variables and the scopes in which these variables are visible.
4. Generate TPL bytecode from the enriched AST. Code generation from the AST to TPL occurs in a single step. In comparison to query code generation, UDF code generation is relatively simple because we lower from an imperative source language (PL/pgSQL) to an imperative target language (TPL). The impedance mismatch between SQL and TPL introduces many challenges that are not present in PL/pgSQL to TPL compilation.

5.2 TPL Closures

One of the most challenging aspects of integrating UDFs with the existing NoisePage execution engine is handling the interaction between the TPL code we generate for the PL/pgSQL function and the associated SQL logic. This dependency manifests in two distinct directions:

- The PL/pgSQL function is invoked by an enclosing query
- The PL/pgSQL function contains embedded SQL query logic

```
CREATE FUNCTION proc_count(x INTEGER)
    RETURNS INTEGER AS $$
DECLARE
    c INTEGER;
BEGIN
    c = (SELECT COUNT(*) FROM foo);
    RETURN c;
END
$$ LANGUAGE PLPGSQL;
```

Listing 3: A UDF with embedded SQL aggregation.

```
CREATE FUNCTION proc_forloop(x INTEGER)
    RETURNS INTEGER AS $$
DECLARE
    tx INTEGER;
    ty INTEGER;
BEGIN
    FOR x, y in (SELECT x, y FROM points) LOOP
        tx += x;
        ty += y;
    END LOOP;
    RETURN tx + ty;
END
$$ LANGUAGE PLPGSQL;
```

Listing 4: A UDF with For-S Style `for`-loop that iterates over query result.

In this section we address our approach to solving the latter problem: how to handle SQL query logic that is embedded in the body of a PL/pgSQL function during code generation. The syntax of the PL/pgSQL language supports two distinct ways in which embedded query logic may be added to a PL/pgSQL function. The first of these is a simple embdded query wherein the query produces only a single result row. In this case, we must assign the results of the query to variables within the body of the PL/pgSQL function. An example of a function that demonstrates this scenario is shown in listing 3.

Users may also embed SQL logic that returns many result rows into the body of a PL/pgSQL function. To support such cases, PL/pgSQL defines a `for`-loop construct that iterates over query result rows. An example of a function that contains such a construct is shown in listing 4.

Handling embedded query logic is complicated by the fact that we would like to leverge all of the existing NoisePage code generation infrastructure to generate code for embedded queries, rather than rewriting the SQL code generation system from scratch or adding special-cases to handle UDFs. To handle both cases of embedded SQL logic while observing this constraint, we extend the TPL language with `closures`. In programming languages, a closure is a generalization of a function that is capable of capturing the environment in which it is constructed. We use closures to bridge the gap between the body of a PL/pgSQL function and its embedded SQL logic.

In NoisePage, results are typically written to a result buffer that is then either prepared for local output (writing to standard output) or serialization to a network buffer. We generalize this functionality by

```
auto udf_body() -> void {
  int count;
  auto closure = [&count](int c) -> void {
      // Assign result to capture variable
      count = c;
  };
  // Invoke query with output callback
  query(state, closure)
}
```

Listing 5: The C++ representation of the closure generated to handle embedded queries with only a single result row.

```
auto udf_body() -> void {
  int tx;
  int ty;
  auto closure = [&tx, &ty](int x, int y) -> void {
      // Assign result to capture variable
      tx += x;
      ty += y;
  };
  // Invoke query with output callback
  query(state, closure)
}
```

Listing 6: The C++ representation of the closure generated to implement a `for`-loop over embedded query result rows.

implementing support for an arbitrary *output callback*. This output callback is invoked by the execution engine for each result row in the query. We combine this output callback functionality with TPL closures to make query results available to arbitrary imperative logic within the body of a PL/pgSQL function.

We model our implemenation of TPL closures off of their analog in the C++ programming language. In listing 5 we provide an example of the interaction between variables declared in the body of the UDF, the closure generated for the query's output callback, and invocation of the query itself. The example is shown in C++ syntax; the generated TPL bytecode is equivalent. Similarly, in listing 6 we demonstrate how a closure is used to implement the PL/pgSQL `for`-loop construct over query result rows.

5.3 System Integration

With the implementation of the PL/pgSQL compiler and the extension of the TPL language with closures, integrating UDF compilation with the existing NoisePage execution engine is straightforward.

As an example consider the UDF definition shown in listing 7 and the associated invocation in the SQL query shown in listing 8. To compile and execute this query, we perform the following steps:

1. Compile the PL/pgSQL function **procCount** to TPL byetcode with our PL/pgSQL compiler. Store the compiled bytecode in the system catalog with a unique identifier.
2. Bind the invocation of **procCount** in the body of the invoking query to the bytecode object stored in the system catalog.
3. Generate TPL bytecode for the invoking query.
4. Clone the AST of the TPL bytecode for the function **procCount** into the bytecode module for the invoking query.

```
CREATE FUNCTION procCount()
    RETURNS INTEGER AS $$
DECLARE
    c INTEGER;
BEGIN
    c := (SELECT COUNT(*) FROM foo);
    RETURN c;
END
$$ LANGUAGE PLPGSQL;
```

Listing 7: Defintion of UDF **procCount**.

```
SELECT procCount();
```

Listing 8: The invocation of UDF **procCount**.

5. Pass this unified bytecode program to the LLVM engine for native JIT-compilation.

System integration brings compilation and execution of PL/pgSQL UDFs to NoisePage.

6. EVALUATION

We now present the results of our evaluation of UDF runtime optimization techniques. We compare UDF JIT-compilation with several baselines, including interpretation-based UDF execution in PostgreSQL and the inlined SQL queries produced by the *Apfel* framework. For our experiments, all benchmark systems are run on Ubuntu Server 18.04. The machine was equipped with Intel(R) Xeon(R) Bronze 3104 CPU @ 1.70GHz (1 processor, 6 cores), 16 GB of RAM and SSD-backed storage.

6.1 Function Call Overhead

The first aspect of our implementation we evaluate is the overhead imposed by individual function calls. While this does not reflect performance improvement or degradation on production workloads, it does allow us to quickly estimate the viability of our approach to UDF optimization via JIT compilation.

In order to measure the runtime overhead imposed by an individual function call, we design a simple synthetic benchmark *Call*. This benchmark consists of three queries:

- A baseline query that performs a sequential scan over a single integer column of a table. The complete text of this query is shown in listing 9.
- A query that invokes the UDF **addOne** to increment this integer column by the constant value one. The definition of **addOne** is shown in listing 10 and the associated invocation is shown in listing 11.
- A query that represents a "perfectly-inlined" version of the call to UDf **addOne**. In this query, the logic to increment the integer column is completely inlined into the top-level query, eliding the function call entirely. The complete text of this query is shown in figure 12.

We run this performance benchmark on two systems: PostgreSQL 12 and NoisePage. Here, PostgreSQL serves as a baseline against which we compare our implementation. PostgreSQL implements

```
SELECT x FROM integers;
```

Listing 9: The baseline query for evaluating the overhead of a UDF call.

```
CREATE FUNCTION addOne(x INTEGER)
    RETURNS INTEGER AS $$
BEGIN
    RETURN x + 1;
END
$$ LANGUAGE PLPGSQL;
```

Listing 10: The definition of UDF **addOne**.

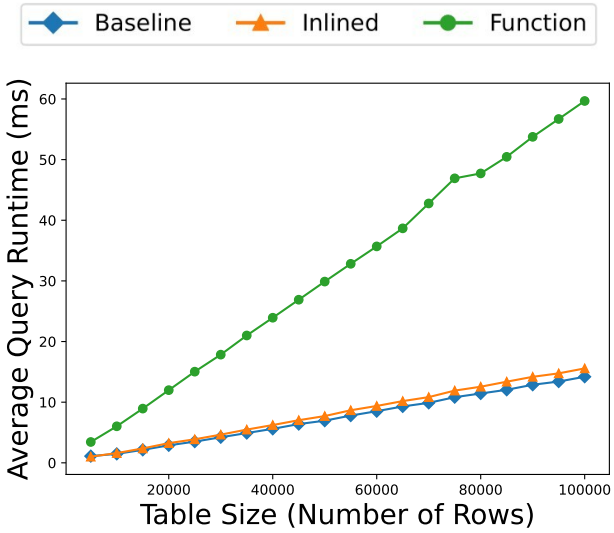


Figure 4: **Function Call Overhead (PostgreSQL)** – Comparison among baseline query, inlined column increment, and column increment with a function call to UDF **addOne** in PostgreSQL.

UDF execution in a manner that incurs significant context-switching overhead [14]. As a consequence of this, we expect to observe high function-call overhead in PostgreSQL. In contrast, the benchmark run on NoisePage uses our approach to UDF JIT-compilation that avoids context switching overhead by virtue of the fact that all of the logic is expressed as a single program in our internal DSL.

The results of this benchmark are shown in figure 4 and figure 5. The results demonstrate the expected trend: UDF execution in PostgreSQL is slower in absolute terms than UDF execution in NosiePage. More importantly, as the size of the table grows, the overall query runtime for PostgreSQL scales much more poorly than it does in NoisePage. This indicates that our implementation of UDF JIT-compilation does not incur large function call overhead, relative to the "optimal" inlined version of the query.

```
SELECT x, addOne(x) FROM integers;
```

Listing 11: The inlined query for evaluating the overhead of a UDF call.

```
SELECT x, x + 1 FROM integers;
```

Listing 12: The inlined query for evaluating the overhead of a UDF call.

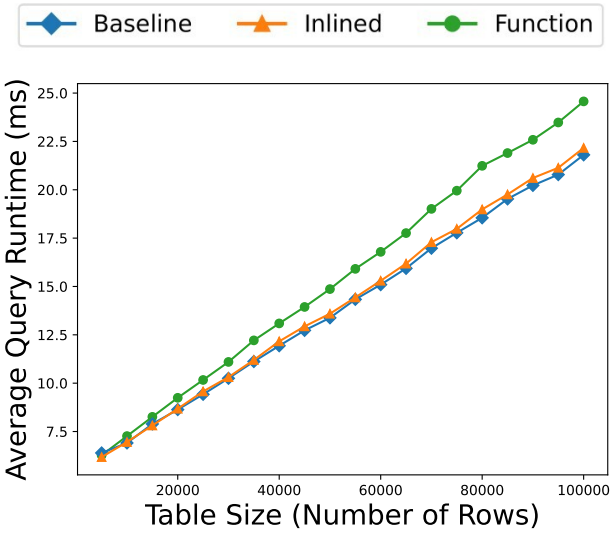


Figure 5: **Function Call Overhead (NoisePage)** – Comparison among baseline query, inlined column increment, and column increment with a function call to UDF **addOne** in NoisePage.

6.2 Lighweight Loops

We construct a second benchmark, *Loop*, to evaluate the viability of our approach to UDF JIT-compilation against the PostgreSQL baseline. In this benchmark, we evaluate the total runtime required for the UDF execution engine to execute a tight loop for a specified number of iterations. Like the previous benchmark, this benchmark does not serve as an indicator of performance on production workloads, but rather demonstrates the potential of our approach to remove software overheads via JIT-compilation of functions.

The definition of the **busyLoop** function is provided in listing 13. This function is invoked with arguments ranging from 10-100,000,000 specifying the number of iterations of the loop executed within the function's body. The results of the benchmark are shown in figure 6. We see that as the number of iterations grows by a factor of 10, increasing from 10,000,000 to 100,000,000, the total query runtime on PostgreSQL explodes as a result of the relatively-high overhead of individual iterations. In contrast, the growth in runtime for NoisePage is much more modest, suggesting that our approach to UDF JIT-compilation elides much of the overhead imposed by interpreter-based systems like PostgreSQL.

6.3 Production Simulation

Finally, we perform an evaluation on selected functions from the SQL ProcBench [20] benchmark. SQL ProcBench is a new benchmark designed to emulate the use of SQL procedural extensions (PL/SQL) in production workloads. The benchmark consists of 28 scalar user-defined functions. We select five of these functions to evaluate because they contain constructs that NoisePage is capable of compiling and that the *Apfel* framework is able to inline. Benchmarks are run on a database with the complete ProcBench schema at scale factor 1.

The first evluation we perform measures the total time required

```
CREATE FUNCTION busyLoop(x INTEGER)
    RETURNS INTEGER AS $$
DECLARE
    c INTEGER := 0;
BEGIN
    WHILE c < x LOOP;
        c = c + 1;
    END LOOP;
    RETURN c;
END
$$ LANGUAGE PLPGSQL;
```

Listing 13: The definition of UDF **busyLoop**.

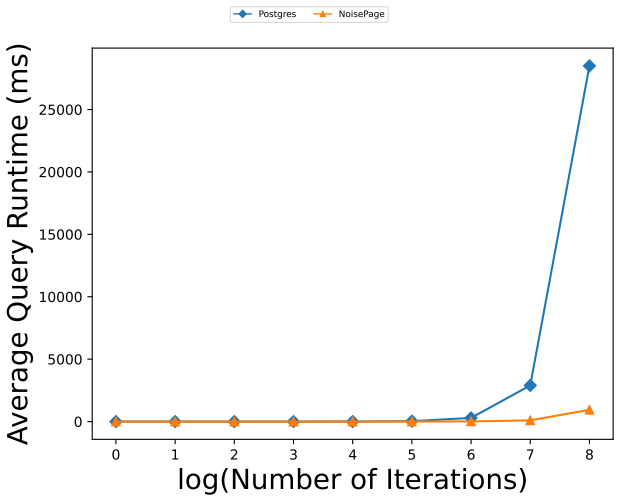


Figure 6: **Function Call Scalability** – Comparison of the runtime required by PostgreSQL and NoisePage to execute the UDF **busyLoop** for a specified number of iterations.

to execute each of functions in the benchmark. We execute each of the five benchmark functions on NoisePage and compare the total runtime against equivalent evaluations on PostgreSQL 12. Unlike the synthetic benchmarks, this benchmark reflects the complete end-to-end performance characteristics of the tested system, rather than the performance characteristics of the UDF execution subsystem in isolation. Results of this benchmark are shown in figure 7.

Unlike the previous two benchmarks, the results of this benchmark are far from inspiring. NoisePage achieves runtimes similar to PostgreSQL for several of the functions (function identifiers **04**, **11**, and **20a**), but far underperforms PostgreSQL on the remaining functions (function identifiers **01** and **05**). The distinction between these two sets of functions is that functions in the former group do not interact heavily with the NoisePage storage subsystem, while functions in the latter group do. As a specific example, the definition of the UDF with function identifier **05** is shown in listing 14. The logic in this function is not complex, but it does involve two aggregations over the result of a three-way **JOIN** on three relatively-large tables, implying that most of the work for this query occurs in the storage subsystem as tables are scanned and the **JOIN** predicates are applied. The NoisePage storage subsystem has several design deficiencies that render this bottleneck unavoidable.

NoisePage's poor performance on these benchmark functions demonstrates the necessity of conducting our final evaluation within

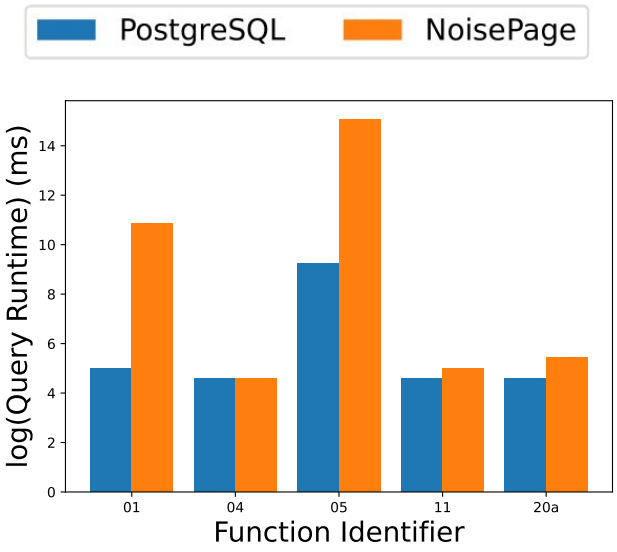


Figure 7: **ProcBench Results - PostgreSQL Baseline** – Comparison of query runtime on PostgreSQL and NoisePage during execution of select functions from the SQL ProcBench benchmark.

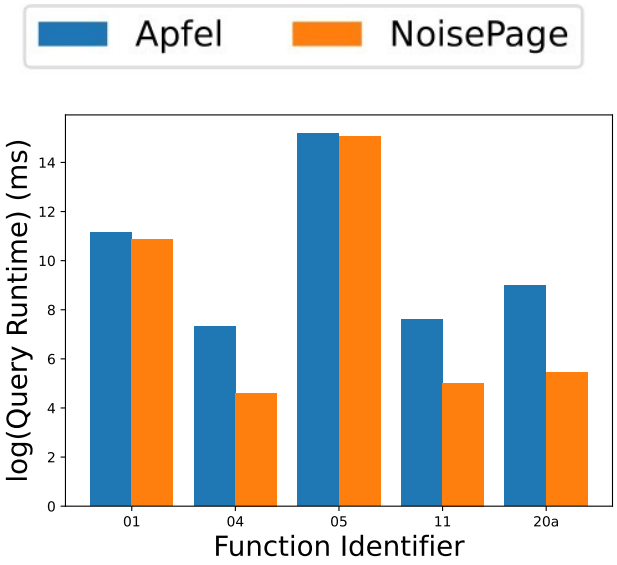


Figure 8: **ProcBench Results - PL/pgSQL Compilation vs Inlining** – Comparison of query runtime on NoisePage during execution of select functions from the SQL ProcBench benchmark.

the context of a single system. Doing so controls for many of the differences among systems so that we may isolate the performance characteristics of UDF optimization strategies.

To this end, we run a second evaluation on the SQL ProcBench benchmark functions. This time, we compare the performance of UDF JIT-compilation in NoisePage with the inlined versions of the benchmark queries produced by the *Apfel* framework. To produce these inlined queries, we compile the original ProcBench functions and their invoking queries with the *Apfel* framework. We then transcribe the SQL queries produced by the framework into query plans that are executed on NoisePage. We must construct each of

```
CREATE FUNCTION morningToEveRatio(dep INT)
    RETURNS INT AS $$
DECLARE
    morningSale INT;
    eveningSale INT;
    totalSale INT;
BEGIN
    morningSale := (SELECT COUNT(*)
        FROM web_sales,
             time_dim,
             customer_demographics
        WHERE ws_sold_time_sk = t_time_sk
            AND ws_bill_customer_sk = cd_demo_sk
            AND t_hour >= 8 AND t_hour <= 9
            AND cd_dep_count = dep);

    eveningSale := (SELECT COUNT(*)
        FROM web_sales,
             time_dim,
             customer_demographics
        WHERE ws_sold_time_sk = t_time_sk
            AND ws_bill_customer_sk = cd_demo_sk
            AND t_hour >= 19 AND t_hour <= 20
            AND cd_dep_count = dep);

    totalSale := morningSale + eveningSale;
    RETURN totalSale;
END;
$$ LANGUAGE PLPGSQL;
```

Listing 14: The definition of ProcBench UDF **05**.

these query plans by hand because NoisePage does not support some of the SQL constructs that appear in the queries produced by *Apfel*. The results of this evaluation are shown in figure 8.

The results of this evaluation are inconclusive. For simpler queries, direct PL/pgSQL compilation provides better performance. This is unsurprising considering that the inlined queries from *Apfel* must model even simple control flow with SQL **JOIN** operations and common table expressions. The NoisePage optimizer is not able to optimize away some of these inefficiencies because the query plans are input directly. For more complex queries (function identifiers **01** and **05**) any performance differences between the two approaches are hidden by the inefficiencies in the NoisePage storage engine.

7. FUTURE WORK

While we completed our intended evaluation in this project, many promising directions for future work remain. We explore several of these areas in the following section.

7.1 Removing Serial Bottlenecks

Our use of TPL closures to bridge the gap between the code generation systems for PL/pgSQL and SQL is general and parsimonious. This design enabled us to quickly solve this engineering problem and continue working on other components required for UDF support. However, this design also imposes a major performance limitation on the system. This design inherently serializes invocation of embedded query logic and disables many of the performance optimizations present in the remainder of the NoisePage execution engine that rely on parallelization. A more general solution to this problem might remove this serial bottleneck and allow distinct invocations of embedded query logic to proceed in parallel.

```
CREATE FUNCTION proc_exception(
        table  VARCHAR,
        c_user INT,
        c_date DATE)
    RETURNS INT AS $$
DECLARE
    x INT;
    y INT;
BEGIN
    INSERT INTO person(first, last) VALUES('Tom', 'Jones');
    BEGIN
        UPDATE person SET first = 'Joe'
            WHERE last = 'Jones';
        x := 1;
        y := x / 0;
    EXCEPTION
        WHEN division_by_zero THEN
            RAISE NOTICE 'caught division_by_zero';
            RETURN x;
        END;
    END;
    RETURN 0;
END;
$$ LANGUAGE PLPGSQL;
```

Listing 15: An example of a UDF containing exceptional control flow.

7.2 Non-Inlinable Constructs

Our PL/pgSQL compiler supports roughly the same set of PL/pgSQL language constructs as the *Apfel* and *Froid* frameworks. There remain some PL/pgSQL constructs, however, that these mature frameworks do not support. Examples of such unsupported constructs include table modifications, exceptional control flow (listing 15) and dynamic SQL (listing 16). Adding support for these constructs may be difficult or even impossible for *Froid* and *Apfel* because of the limitations imposed by inlining at the SQL level. We foresee no such limitations in our approach.

7.3 Decoupling Inlining from Execution

Throughout much of this project, we conceived of our approach to UDF runtime optimization as fundamentally distinct from the inlining approach used in the *Apfel* and *Froid* project. However, the primary difference between these two approaches is the level of abstraction at which the inlining is performed. Both *Apfel* and *Froid* perform UDF inlining at the SQL level, producing a query that implements the logic of both the inlined function and the original invoking query. In NoisePage, we compile both the function and the invoking query to our own internal intermediate representation, TPL bytecode, and combine the AST representation of this bytecode to produce a unified program that implements the query. Both approaches solve the UDF runtime overhead problem by inlining the logic the UDF and removing the necessity of performing a context switch to execute imperative logic.

We find in our evaluation that inlining at the SQL level and inlining at the level of the TPL DSL result in similar performance characteristics. However, SQL inlining does present some additional benefits beyond TPL inlining.

```
CREATE FUNCTION proc_dynamic(
        table  VARCHAR,
        c_user INT,
        c_date DATE)
    RETURNS INT AS $$
DECLARE
    c INT;
BEGIN
    EXECUTE 'SELECT COUNT(*) FROM '
        || quote_ident(table)
        || ' WHERE istd_by = $1 AND istd <= $2'
    INTO c
    USING checked_user, checked_date;
    RETURN c;
END;
$$ LANGUAGE PLPGSQL;
```

Listing 16: An example of a UDF containing dynamic SQL.

The first of these benefits is the engineering cost of the solution. The authors of *Froid* claim that the original version of the framework was implemented in just 1,500 lines of code within Microsoft SQL Server [27] In contrast, our PL/pgSQL compiler, extensions to the TPL language, and integration with the rest of the NoisePage execution engine required the addition of some 10,000 lines of code. The *Froid* engineers were able to implement their solution with far fewer additions because the system operates at a much higher level of abstraction.

Beyond engineering overhead, SQL inlining also provides portability. The *Froid* framework is implemented within Microsoft SQL Server. Currently it does not provide an interface through which inlined queries can be extracted directly from the system [27], but it is possible to determine the structure of the inlined query plans produced by *Froid* with the help of SQL's **EXPLAIN** command. The *Apfel* framework is implemented as standalone compiler [14]. The authors even provide a web interface through which arbitrary users may submit PL/pgSQL functions and invoking queries and observe how the framework performs inlining [2]. In contrast, our implementation of UDF inlining with NoisePage is tightly coupled to the remainder of the NoisePage execution engine. The fact that our representation of the unified query plan utilizes our internal DSL renders the technique non-portable.

Finally, SQL inlining presents the opportunity to leverage decades of work on query optimization to produce efficient inlined query plans. Both *Froid* and *Apfel* are capable of producing pure-SQL representations of the inlined query, and this representation may subsequently be run through a standard SQL query optimizer to improve its performance. In NoisePage, we perform inlining *after* query optimization is applied to the invoking SQL query. This prevents us from capturing the benefits of optimization that may only become visible to the optimizer after the query has been inlined.

To account for the strengths and limitations of both approaches, we propose decoupling UDF inlining from the execution environment. This would allow us to take advantage of all of the aforementioned benefits of SQL inlining within the context of a compiled query engine like NoisePage.

8. CONCLUSION

UDFs allow us express SQL queries more efficiently and in a manner that encourages modularization and reuse. They make data management in production safer and more efficient. However, their use in production systems is often discouraged because of their poor performance characteristics. This poor performance is a result of the impedance mismatch between declarative SQL and imperative PL/SQL which often necessitates frequent, expensive context-switches during query execution.

In this paper, we address this problem using a novel approach to UDF runtime optimization that integrates with a query compilation engine. In our research system, NoisePage, we implement a compiler for PL/pgSQL UDFs that targets the same intermediate representation as our existing execution engine architecture. We combine the code generated for both the UDF and its invoking query into a complete program and JIT-compile this program to native machine code with LLVM. In an experimental evaluation we find that our approach yields performance that is competitive with state-of-the-art approaches to UDF optimization through inlining.

9. ACKNOWLEDGEMENTS

I would like to thank Karthik Ramachandra from Microsoft. His pioneering work on *Froid* and consistent willingness to discuss its inner workings enabled me to complete my evaluation in this project.

I would also like to thank Torsten Grust and Denis Hirn from the University of Tubingen. This project would not have been possible were it not for their willingness to provide insights on their *Apfel* framework and the means to utilize it with a custom configuration.

I would like to thank my Capstone advisor Andrew Pavlo for his consistent feedback and enthusiasm for this project. There were many times over the course of this project that I felt discouraged by new challenges and slow progress. Andy always managed to reignite my passion for this work.

Finally, I would like to thank all of my colleagues in the Carnegie Mellon Database Group for constantly inspiring me to strive for database-greatness. Specifically, I must thank former member Tanuj Nayak. Tanuj's work on PL/pgSQL compilation forms the foundation for this project; I would not have been able to complete it were it not for his example.

10. REFERENCES

[1] Apache spark. https://spark.apache.org/.

[2] Apfel web interface. http://apfel-db.informatik.uni-tuebingen.de/.

[3] HyPer. http://hyper-db.de/.

[4] LLVM. http://llvm.org.

[5] Microsoft SQL Server. https://www.microsoft.com/en-us/sql-server/.

[6] NoisePage. https://noise.page.

[7] PL/pgSQL. https://www.postgresql.org/docs/13/plpgsql.html.

[8] Postgresql. https://www.postgresql.org/.

[9] SingleStore. https://www.singlestore.com/.

[10] The GNU Compiler Collection. https://gcc.gnu.org/.

[11] T. N. Andre Kohn, Viktor Leis. Adaptive execution of compiled queries, 2018.

[12] K. D. T. K. U. C. S. Z. Andrew Crotty, Alex Galakatos. Tupleware: Redefining modern analytics, 2014.

[13] D. D. Chamberlin, M. M. Astrahan, M. W. Blasgen, J. N. Gray, W. F. King, B. G. Lindsay, R. Lorie, J. W. Mehl, T. G. Price, F. Putzolu, P. G. Selinger, M. Schkolnick, D. R. Slutz, I. L. Traiger, B. W. Wade, and R. A. Yost. A history and evaluation of system r. 1981.

[14] C. Duta, D. Hirn, and T. Grust. Compiling pl/sql away, 2019.

[15] C. Freedman, E. Ismert, and P.-A. Larson. Compilation in the microsoft SQL server hekaton engine. *IEEE Data Eng. Bull.*, 2014.

[16] G. Graefe. Query evaluation techniques for large databases, 1993.

[17] G. Graefe. Volcano: An extensible and parallel query evaluation system, February 1994.

[18] J. M. D. K. K. B. K. O. Gregory M. Essertel, Ruby Y. Tahboub and T. Rompf. Flare: Optimizing apache spark with native compilation for scale-up architectures and medium-size data, 2018.

[19] S. Gupta, S. Purandare, and K. Ramachandra. Optimizing cursor lookps in relational databases, 2020.

[20] S. Gupta and K. Ramachandra. Procedural extensions of sql: Understanding their usage in the wild, 2021.

[21] A. Kemper and T. Neumann. HyPer: A hybrid OLTP&OLAP main memory database system based on virtual memory snapshots. In *ICDE*, pages 195–206, 2011.

[22] K. Krikellas, S. D. Viglas, and M. Cintra. Generating code for holistic query evaluation. In *Data Engineering (ICDE), 2010 IEEE 26th International Conference on*, pages 613–624. IEEE, 2010.

[23] M. S. Leonhard Spiegelberg, Rahul Yesantharao and T. Kraska. Tuplex: Data science in python at native code speed, 2021.

[24] P. Menon, A. Ngom, L. Ma, T. C. Mowry, and A. Pavlo. Permutable compiled queries: Dynamically adapting compiled queries without recompiling, 2020.

[25] T. Neumann. Efficiently compiling efficient query plans for modern hardware. *PVLDB*, 4(9):539–550, 2011.

[26] D. Paroski. Code generation: The inner sanctum of database performance. http://highscalability.com/blog/2016/9/7/code-generation-the-inner-sanctum-of-database-performance.html, September 2016.

[27] K. Ramachandra, K. Park, K. V. Emani, A. Halverson, C. Galindo-Legaria, and C. Cunningham. Froid: Optimization of imperative programs in a relational database, 2019.

[28] D. L. Sameer Agarwal and R. Xin. Apache spark as a compiler: Joining a billion rows per second on a laptop. https://databricks.com/blog/2016/05/23/apache-spark-as-a-compiler-joining-a-billion-rows-per-second-html, 2016.

[29] A. Silberschatz, H. F. Korth, and S. Sudarshan. *Database System Concepts*. McGraw-Hill, seventh edition edition, 2019.